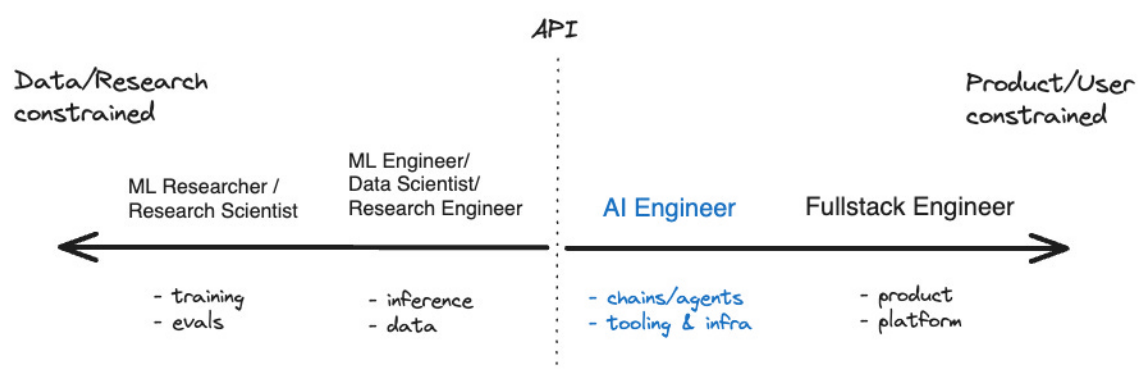


The Rise of the AI Engineer

Emergent capabilities are creating an emerging title: to wield them, we'll have to go beyond the Prompt Engineer and write *software*, and AI that writes software.

We are observing a once in a generation "shift right" of applied AI , fueled by the emergent capabilities and open source/API availability of Foundation Models.

A wide range of AI tasks that used to take 5 years and a research team to accomplish in 2013 , now just require API docs and a spare afternoon in 2023 .



“ “In numbers, there's probably going to be significantly more AI Engineers than there are ML engineers / LLM engineers. One can be quite successful in this role without ever training anything.” - Andrej Karpathy

“In numbers, there's probably going to be significantly more AI Engineers than there are ML engineers / LLM engineers. One can be quite successful in this role without ever training anything.” - Andrej Karpathy

However, the devil is in the details - there are no end of challenges in successfully evaluating, applying and productizing AI:

Models: From evaluating the largest GPT-4 and Claude models, down to the smallest open source Huggingface, LLaMA, and other models

Tools : From the most popular chaining, retrieval and vector search tools like LangChain, LlamaIndex, and Pinecone to the emerging field of autonomous agents like Auto-GPT and BabyAGI (must-read recap from Lilian Weng here)

Research/Progress : On top of this, the sheer volume of papers and models and techniques published each day is exponentially increasing with interest and funding, so much so that keeping on top of it all is almost a full time job.

I take this seriously and literally. I think it is a full time job . I think software engineering will spawn a new subdiscipline, specializing in applications of AI and wielding the emerging stack effectively, just as “ site reliability engineer ”, “ devops engineer ”, “ data engineer ” and “ analytics engineer ” emerged.

The emerging (and least cringe) 1 version of this role seems to be: AI Engineer .

Every startup I know of has some kind of #discuss-ai Slack channel. Those channels will turn from informal groups into formal teams, as Amplitude , Replit and Notion have done. The thousands of Software Engineers working on productionizing AI APIs and OSS models, whether on company time or on nights and weekends, in corporate Slacks or indie Discords, will professionalize and converge on a title - the AI Engineer. This will likely be the highest-demand engineering job of the decade.

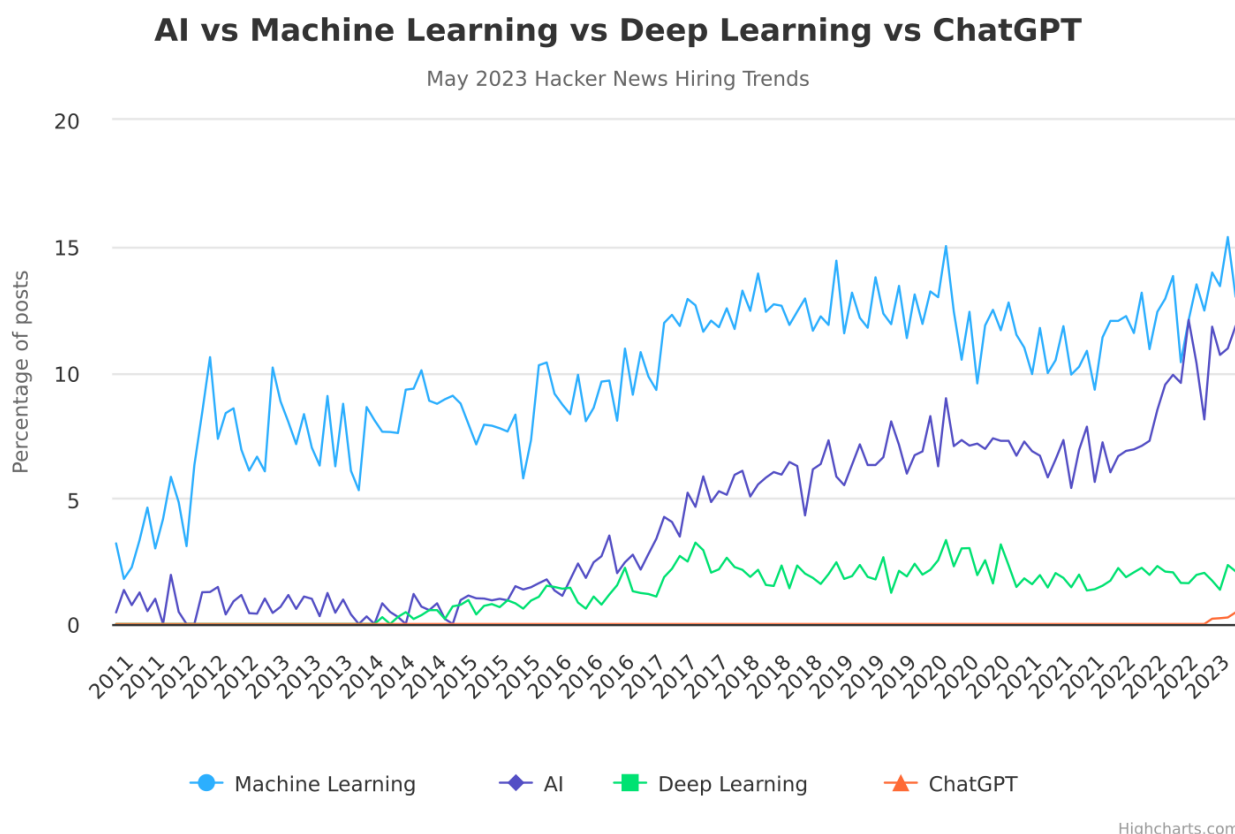
AI Engineers can be found everywhere from the largest companies like Microsoft and Google, to leading edge startups like Figma (via Diagram acquisition), Vercel (eg Hassan El Mghari's viral RoomGPT) and Notion (eg Ivan

Zhao and Simon Last with Notion AI) to independent hackers like Simon Willison , Pieter Levels (of Photo/InteriorAI) and Riley Goodside (now at Scale AI). They are making \$300k/yr doing prompt engineering at Anthropic and \$900k building software at OpenAI . They are spending free weekends hacking on ideas at AGI House and sharing tips on /r/LocalLLaMA 2 . What is common among them all is they are taking AI advancements and shaping them into real products used by millions, virtually overnight.

Not a single PhD in sight. When it comes to shipping AI products, you want engineers, not researchers .

The AI vs ML Engineer Flipping

I am calling attention to this trend rather than starting it. There are 10x as many ML Engineer jobs as AI Engineer jobs on Indeed, but the higher growth rate of “AI” leads me to predict that this ratio will invert in 5 years.



All job titles are flawed, but some are useful. We are both wary and weary of the endless semantic debates on the difference between AI and ML, and are well aware that regular “software engineer” roles are perfectly capable of building AI software. However a recent Ask HN question on How to Break into AI Engineering illustrates the fundamental perception that still persists in the market:

▲ Ask HN: How to Break into AI Engineering

126 points by dragonmouse 6 days ago | flag | hide | past | archive | favorite | 64 comments | toggle all comments

What are some great resources for learning the skills and knowledge necessary to start a career as an AI engineer?

Gracias!

▲ __rito__ 6 days ago | flag | favorite | next [-]

- Have crystal clear Mathematical foundations, as in why this formula/method the way it is, rather than being able to solve college/HS test problems. Really solid footing in Differential Calculus and Linear Algebra is necessary.
- Know the Statistical language that you learn from a basic college-level Stat 101 course. Be able to translate normal sentences into those using Statistical notation, and be able to read easily. Also, know basic Statistics.
- You already know programming, I assume. Learn Python if you don't know already. It's really easy.
- There are a number of paths you can go from there. Here's what I did.
- IBM Data Science Professional Certificate (not deep at all, but lays out the landscape well; did it in a week)
- Machine Learning for Absolute Beginners by Oliver Theobald which you can finish in an evening.
- Machine Learning Specialization by Andrew Ng on Coursera.
- Deep Learning Specialization by Andrew Ng on Coursera.
- fast.ai course.
- Learn PyTorch really well. I suggest Sebastian Raschka's book.

Now from here, you can chart your own path. You can choose NLProc, Vision, RL, or something else.

I went towards Vision. And I do Edge AI as hobby.

I was in the last year of college as a Physics undergrad, when I was hired to do Vision modelling/research for a non-flashy company in 2021. Finishing my CS Master's next month and starting to look for PhD. I worked in the same company for the ~2.5 years.

EDIT: If you want a job in big tech, grind Leetcode, and learn about system design, study Machine Learning systems, and be able to design them. Chip Huyen has a good book as I hear. 6-7 rounds of interview is common in Meta/Google. DL hackathon awards, open source contributions are significantly helpful.

[reply](#)

jahsome 6 days ago | flag | favorite | parent | next [12 more] [collapse root]

maypop 6 days ago | flag | favorite | parent | prev | next [1 more] [collapse root]

▲ santiagobasulto 6 days ago | flag | favorite | prev | next [-]

AI Engineering is basically Data Engineering focused on AI. When in "traditional" Data Engineering you create pipelines that store processed data in something like a Data Lake, in AI Eng. your end storage might be a specialized Feature Storage (like Feast or GCP Vertex AI).

There are some AI Engineers with strong scientific/mathematical background, but that's rare. Usually, you're paired with these ML people that actually develop and evaluate the models.

So my advice is to start with Data Engineering and then find a specialization AI. You should have a VERY solid foundation on scripting and programming, specially Python. Also, a lot of concepts of "data wrangling". Understanding how data flows from point A to point B, how the intermediate storages and streaming engines work, etc. Functional programming is key here.

[0] <https://github.com/feast-dev/feast>

[reply](#)

▲ jstx1 6 days ago | flag | favorite | parent | next [-] [collapse root]

> AI Engineering is basically Data Engineering focused on AI.

I work in machine learning and this isn't how I see it at all. Data engineering specifically evolved as a term to differentiate the people who work with data but don't work on ML/AI.

[reply](#)

Most people still consider AI Engineering as a form of either Machine Learning or Data Engineering, so they recommend the same prerequisites. But I guarantee you that none of the highly effective AI Engineers I named above have done the equivalent work of the Andrew Ng Coursera courses, nor do they know PyTorch, nor do they know the difference between a Data Lake or Data Warehouse 3.

In the near future, nobody will recommend starting in AI Engineering by reading Attention is All You Need, just like you do not start driving by reading the schematics for the Ford Model T. Sure, understanding fundamentals and history is always helpful, and does help you find ideas and efficiency/capability gains that are not yet in common consciousness. But sometimes you can just use products and learn their qualities through experience.

I don't expect this "flipping" of the curriculum to happen overnight. It is human nature to want to stuff a resume, fill out a market map, and stand out by citing deeper topics with more authority. In other words, Prompt Engineering and AI Engineering will feel inferior to people with good Data Science/ML backgrounds for a long while. However, I think sheer demand-and-supply economics will prevail.

Why AI Engineers are Emerging Now

Foundation Models are "few shot learners", exhibiting in-context learning and even zero shot transfer capabilities that generalize beyond the original intent of model trainers. In other words, the people creating the models don't fully know what they are capable of. People who aren't LLM researchers are able to find and exploit capabilities simply by spending more time with the models, and applying them to a domain that is undervalued by research (e.g. Jasper with copywriting).

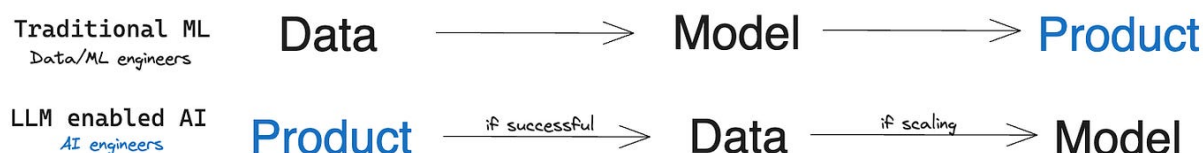
Microsoft, Google, Meta, and the large Foundation Model labs have cornered scarce research talent to essentially deliver "AI Research as a Service" APIs. You can't hire them, but you can rent them — if you have software engineers on the other end who know how to work with them. There are ~5000 LLM researchers in the world, but ~50m software engineers. Supply constraints dictate that an "in-between" class of AI Engineers will rise to meet demand.

GPU hoarding. Of course OpenAI/Microsoft was first, but Stability AI kicked off the startup GPU arms race by emphasizing their 4,000 GPU cluster.



Since then it has become commonplace for new startups like Inflection (\$1.3b), Mistral (\$113m), Reka (\$58m), Poolside (\$26m) and Contextual (\$20m) to raise huge seed rounds in order to own their own hardware. Dan Gross and Nat Friedman even announced Andromeda, their \$100m, 10 exaflop GPU cluster exclusively for startups they invest in. The global chip shortage is reflexively creating even more shortage. There will be much more capacity for AI Engineers on the other side of the API line to use models, rather than train them.

Fire, ready, aim. Instead of requiring data scientists/ML engineers do a laborious data collection exercise before training a single domain specific model that is then put into production, a product manager/software engineer can prompt an LLM, and build/validate a product idea, before getting specific data to finetune.



Let's say there are 100-1000x more of the latter than the former, and the “fire, ready, aim” workflow of prompted LLM prototypes lets you move 10-100x faster than traditional ML. So AI Engineers will be able to validate AI products say 1,000-10,000x cheaper. It's Waterfall vs Agile, all over again. AI is Agile.

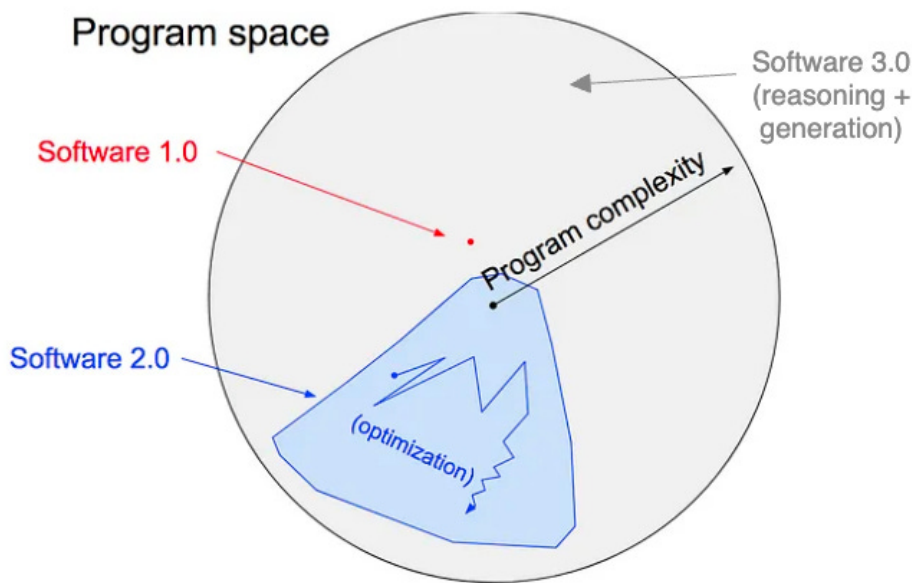
Python + JavaScript . Data/AI is traditionally extremely Python centric, and the first AI Engineering tools like LangChain, LlamaIndex and Guardrails arose out of that same community. However, there are at least as many JavaScript developers as Python developers, so now tools are increasingly catering to this widely expanded audience, from LangChain.js and Transformers.js to Vercel's new AI SDK . The TAM expansion and opportunity is at least 100% bigger.

Generative AI vs Classifier ML . “Generative AI” 7 as a term has fallen out of favor, giving way to other analogies like “ reasoning engine ”, but is still useful in concisely articulating the difference between the existing group of MLOps tools and ML practitioners, and the rising, starkly different kind of persona that is best wielding LLMs and text to image generators. Where the existing generation of ML might have been focused on fraud risk, recommendation systems, anomaly detection, and feature stores , the AI Engineers are building writing apps, personalized learning tools, natural language spreadsheets, and Factorio-like visual programming languages . 8

Whenever a subgroup arises that has a completely different background, speaks a different language, produces a completely different set of products, and uses a completely different set of tools, they eventually split out into their own group.

1+2=3: The Role of Code in the evolution from Software 2.0 to Software 3.0

6 years ago, Andrej Karpathy wrote a very influential essay describing Software 2.0 - contrasting the “classical stack” of hand-coded programming languages that precisely model logic against the new stack of “machine learned” neural networks that approximate logic, enabling software to solve a lot more problems than could humanly be modeled. In 2023, he now notes that the hottest new programming language is English , finally filling out the gray area in his diagram that was left unlabeled in the original essay - moving from Software 2.0 to something... a LOT broader.



“ Update: Andrej responds! with some disagreement !

Update: Andrej responds! with some disagreement !

Last year, Prompt Engineering was the memetic take on how jobs would change as people began to put GPT-3 and Stable Diffusion to work. People derided AI startups as “OpenAI Wrappers”, and fretted as LLM apps proved susceptible to prompt injection and reverse prompt engineering . No moat to be found?

But one of the biggest themes of 2023 has very much been about re-establishing the role of human-written code to orchestrate and supplant LLM power, from the >\$200m behemoth Langchain , to Nvidia-backed Voyager showing the unmistakable importance of code generation and reuse (I recently took part in a Chains vs Agents webinar with Harrison where I expanded on the thesis of Code Core vs LLM Core applications).

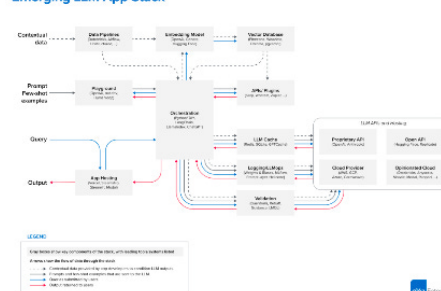
The screenshot shows a presentation slide titled "Code Shell, LLM Core is fundamentally constrained by LLM capabilities". The slide is presented in a Beamer environment with a sidebar on the left showing slide numbers 16 through 21. The main content area is divided into two columns. The left column is titled "LLM Core Code Shell" and lists examples: "e.g. Retrieval Augmented Generation, Chat, Backend-GPT, Marvin AI, AutoGPT". Below this is a box labeled "Code Shell" containing a red circle with a dragon-like creature labeled "LLM Core" and a blue box labeled "Langchain LlamaIndex" with a database icon labeled "Vector DB". The right column is titled "LLM Shell Code Core" and lists examples: "e.g. Copilot, Voyager, Smol-Developer". Below this are four boxes, each containing a red circle with a dragon-like creature and a blue box with a specific function: "COT/TOT" (Planner Agent/ Task Queue), "H2B2 Hypothesize/ Research" (Context Agent/ Theory of Mind), "DraPT/Test" (Skills Library), and "Design System/ User prefs" (UI/Interface Generation Agent).

Image

Prompt Engineering was both overhyped and here to stay, but the re-emergence of Software 1.0 paradigms in Software 3.0 applications is both an area of mass opportunity/confusion, and created white space for a mess of startups:



Emerging LLM App Stack



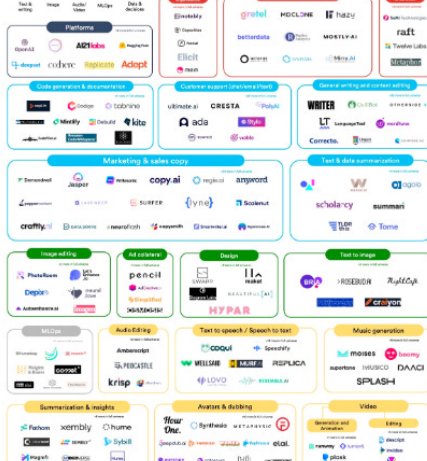
The Generative AI Application Landscape

A work in progress

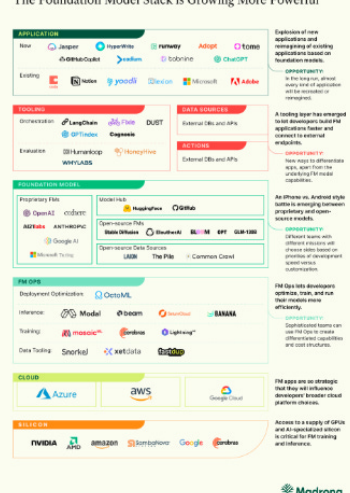


BASE10 TREND MAP: GENERATIVE AI

Companies are grouped based on medium produced and segmented by use case within each medium. Companies that offer products across segments are grouped in the segment of the core product offering.



The Foundation Model Stack is Growing More Powerful



It's not going to be just human-written code, of course. My recent adventures with smol-developer , the larger scoped gpt-engineer , and other code generation agents like Codium AI , Codegen.ai and Morph/Rift will increasingly be a part of the AI Engineer toolkit. As human Engineers learn to harness AI, AIs will increasingly do Engineering as well, until a distant future when we look up one day and can no longer tell the difference.

It's Time To Converge - AI Engineer Summit

Builders need a place to talk turpentine. This is why, after months of organizing small meetups , we are now announcing the first independently run, builder oriented AI conference: The AI Engineer Summit !

AI Engineer SUMMIT

**A technical conference
for next-generation builders.**

October 8 - 10 • San Francisco

If everything in this post is resonating with you, we aim to convene all the top AI Engineers, founders, and investors together to learn about the state of the art, attend/teach workshops and find everything from the great new tool they'll use at work, to their next new hire/cofounder/round.

The definitive conference to discuss everything that we've covered in the past year on this newsletter and our podcast, and more:

AI UX

AI Devtools

AI Infra

AI Agents

New LLM Tools, including Langchain, Vector DBs, and more

Open Source Models (training, finetuning, inferencing, evaling)

I have a fair amount of experience running community, but have never run a 500 person conference, so I have teamed up with Ben Dunphy of Reactathon to put on the best AI Engineer conference in San Francisco (and online - his last conference had 20,000+ tuning in remotely).

Join us at ai.engineer !

We are accepting both speaker CFPs and sponsors (get in touch!).

We are the Builders

Keen observers will have noticed that we've gradually the Latent Space podcast and newsletter to cater to the AI Engineer persona. What excites me most about serving this audience is the combination of techno-optimism and practicality . Marc Andreessen recently wrote about how the vast majority of public AI discourse has been "hysterical fear and paranoia", calling out that there is a whole profession of "AI safety expert", "AI ethicist", "AI risk researcher" paid to be doomers, but no corresponding role for builders and foomers . On the other end of the spectrum, there are many unserious accelerationists and intolerable foomer threadbois who spend all day on

Twitter talking about a distant Utopian future, but it's unclear what they are doing to bring it about.

AI Engineers will tame and ride Shoggoth .

Let's make this a thing.

Thanks for the many comments and questions on HN and Twitter ! We convened a snap Twitter Space to talk it through and >1,000 AI Engineers tuned in . The Rise of the AI Engineer has also been covered in other podcasts .

Author's note : I am especially grateful to my cohost Alessio Fanelli of Decibel and Sarah Guo and Pranav Reddy of Conviction for reviewing drafts of this post and providing critical feedback and invaluable support. And of course, Ben Dunphy for agreeing to cofound the AI Engineer conference series and network. Thank you!

Alternatives considered: "Foundation Model Engineer", "AI API Developer", "LLM Engineer", but it doesn't quite roll off the tongue, whereas "Prompt Engineer" is also too limited for the Code Core, LLM Shell we are seeing emerge. There is also " MLOps Engineer ", but this is a well established community historically focused on lower level ML concerns — a distinction we discuss later in this piece. Lastly, " AI Tinkerer " has been popularized by Alex Gravely ((disputed?) creator of Copilot), but unfortunately doesn't seem titleworthy.

My full list of AI podcasts, newsletters , and communities is maintained on GitHub.

And probably could not explain transformers on a whiteboard .

Stability and AWS' relationship on the cluster is a matter of active debate, so it is best to hear from Emad in his own words on this.

I'm workshopping numeronyms for them. g12n?

More accurately, this is an Nvidia chip shortage. This is why George Hotz's Tinycorp is so important in being able to run popular open source models on AMD cards. Mosaic's LLMFoundry also reports AMD compatibility.

And, thank goodness, "Gen AI".

Of course, when you can combine both disciplines well, you have outlier results like Character.ai , founded by a coauthor on the Transformers paper but finding a great use for the generative aspect of LLMs.

Subscribe to Latent.Space



The Canaanite's avatar



Ryan Nisbet's avatar



Vikram Dutt's avatar



ming's avatar



Oskar Austegard's avatar